

Escuela de Ingeniería
de Sistemas e
Informática

Bases de Datos 2



cassandra

Taller cassandra

Jathinson Meneses M.

✉ Jathinson@gmail.com

Henry Andres Jimenez H.

✉ han315730@gmail.com

Modelado distribuido, optimización y análisis de rendimiento en Cassandra usando Astra DB Cloud

Tecnologías:

- Apache Cassandra

- Astra DB

- CQL (Cassandra Query Language)

Objetivo General: Diseñar e implementar un modelo de datos distribuido orientado a consultas complejas en Cassandra, aplicando principios de desnormalización, modelado por consultas, particionamiento y optimización en entornos cloud usando Astra DB.

Caso de estudio

La universidad desea implementar un sistema académico distribuido capaz de soportar:

- millones de registros,
- consultas concurrentes,
- múltiples sedes,
- almacenamiento histórico,
- alta disponibilidad,
- tolerancia a fallos.

El sistema debe permitir:

- consultar estudiantes por carrera,
- consultar cursos por semestre,
- consultar historial académico,
- consultar materias por periodo,
- consultar estudiantes inscritos por curso,
- consultar rendimiento académico.

Configuración inicial (5 min)

Actividad 1

Ingresar al CQL Console de Astra DB.

```
--Verificar los KEYSPACES  
  
DESCRIBE KEYSPACES;
```

Seleccionar el keyspace:

```
--Seleccionar el keyspace:  
|  
USE default_keyspace;
```

Verificar tablas existentes:

```
--Verificar tablas existentes:  
  
DESCRIBE TABLES;
```

Diseño orientado a consultas (15 min)

Cassandra NO diseña primero relaciones sino consultas

Actividad de análisis

La universidad necesita responder rápidamente:

Consulta	Frecuencia
Estudiantes por carrera	Muy alta
Cursos por semestre	Alta
Historial de notas	Muy alta
Estudiantes inscritos en curso	Muy alta
Promedio académico por estudiante	Alta

usar JOIN sería problemático en Cassandra

Creación de tablas avanzadas

Estudiantes por carrera

```
CREATE TABLE estudiantes_por_carrera (  
  carrera TEXT,  
  id_estudiante UUID,  
  nombre TEXT,  
  correo TEXT,  
  semestre INT,  
  promedio FLOAT,  
  PRIMARY KEY (carrera, id_estudiante)  
);
```

Cursos por semestre

```
CREATE TABLE cursos_por_semestre (  
  semestre INT,  
  id_curso UUID,  
  nombre TEXT,  
  profesor TEXT,  
  credits INT,  
  PRIMARY KEY (semestre, id_curso)  
);
```

Historial académico

```
CREATE TABLE historial_academico (  
  id_estudiante UUID,  
  periodo TEXT,  
  fecha TIMESTAMP,  
  id_curso UUID,  
  curso TEXT,  
  nota FLOAT,  
  estado TEXT,  
  PRIMARY KEY ((id_estudiante, periodo), fecha)  
)  
WITH CLUSTERING ORDER BY (fecha DESC);
```

Estudiantes inscritos por curso

```
CREATE TABLE inscritos_por_curso (  
  id_curso UUID,  
  id_estudiante UUID,  
  nombre TEXT,  
  carrera TEXT,  
  promedio FLOAT,  
  PRIMARY KEY (id_curso, id_estudiante)  
);
```

Rendimiento académico

```
CREATE TABLE rendimiento_estudiante (  
  id_estudiante UUID,  
  semestre INT,  
  promedio FLOAT,  
  materias_aprobadas INT,  
  materias_perdidas INT,  
  PRIMARY KEY (id_estudiante, semestre)  
);
```

Inserción avanzada de datos (15 min)

Insertar:

estudiantes

```
INSERT INTO estudiantes_por_carrera
(carrera, id_estudiante, nombre, correo, semestre, promedio)
VALUES
('Ingenieria de Sistemas', uuid(), 'Ana Torres',
'ana@unab.edu.co', 8, 4.5);

INSERT INTO estudiantes_por_carrera
(carrera, id_estudiante, nombre, correo, semestre, promedio)
VALUES
('Ingenieria de Sistemas', uuid(), 'Carlos Perez',
'carlos@unab.edu.co', 7, 4.1);
```

CURSOS

```
INSERT INTO cursos_por_semestre
(semestre, codigo_curso, nombre_curso, creditos, profesor)
VALUES
(1, 'MAT101', 'Calculo Diferencial', 4, 'Carlos Gomez');

INSERT INTO cursos_por_semestre
(semestre, codigo_curso, nombre_curso, creditos, profesor)
VALUES
(1, 'PRO102', 'Introduccion a la Programacion', 3, 'Laura Rojas');
```


registros académicos

```
INSERT INTO historial_academico
(id_estudiante, periodo, codigo_curso, nombre_curso, nota, estado)
VALUES
(uuid(), '2025-1', 'MAT101', 'Calculo Diferencial', 4.2, 'APROBADO');

INSERT INTO historial_academico
(id_estudiante, periodo, codigo_curso, nombre_curso, nota, estado)
VALUES
(uuid(), '2025-1', 'PRO102', 'Introduccion a la Programacion', 4.8, 'APROBADO');
```

inscripciones

```
INSERT INTO inscritos_por_curso
(codigo_curso, id_estudiante, nombre_estudiante, semestre, estado)
VALUES
('BD401', uuid(), 'Ana Torres', 8, 'ACTIVO');

INSERT INTO inscritos_por_curso
(codigo_curso, id_estudiante, nombre_estudiante, semestre, estado)
VALUES
('BD401', uuid(), 'Carlos Perez', 8, 'ACTIVO');
```

rendimiento_estudiante

```
INSERT INTO rendimiento_estudiante  
(id_estudiante, semestre, promedio, materias_aprobadas, materias_reprobadas)  
VALUES  
(uuid(), 1, 4.5, 5, 0);
```

```
INSERT INTO rendimiento_estudiante  
(id_estudiante, semestre, promedio, materias_aprobadas, materias_reprobadas)  
VALUES  
(uuid(), 2, 3.8, 4, 1);
```

Consultas

- 1 Consultar todos los estudiantes de Ingeniería de Sistemas con promedio superior a 4.0.
- 2 Consultar estudiantes de octavo semestre ordenados por promedio descendente.
- 3 Obtener todos los cursos del semestre 8.
- 4 Consultar cursos con más de 3 créditos.
- 5 Buscar estudiantes inscritos en Bases de Datos II.
- 6 Consultar únicamente estudiantes activos en un curso.
- 7 Obtener estudiantes que reprobaron materias.
- 8 Consultar materias aprobadas con nota superior a 4.5.

- 9 Consultar historial académico de un periodo específico.
- 10 Obtener estudiantes con promedio inferior a 3.0.
- 11 Consultar estudiantes con más materias reprobadas que aprobadas.
- 12 Consultar los mejores promedios académicos.
- 13 Obtener cursos dictados por un profesor específico.
- 14 Consultar estudiantes inscritos en cursos avanzados (semestres 8, 9 y 10).
- 15 Consultar estudiantes con rendimiento sobresaliente.

Link CQL